



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ**



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

**Asistent conversațional pentru proceduri
administrative din România folosind RAG și local
LLMs**

Absolvent

Cosmin-Ștefan Glod

Coordonator științific

Prof. dr. Alin Ștefănescu

București, iunie 2026

Rezumat

Această lucrare prezintă un asistent conversațional destinat cetățenilor români, care răspunde la întrebări despre proceduri administrative (înregistrarea nașterii, încheierea căsătoriei și închirierea unei locuințe), pe baza datelor structurate publicate de proiectul Code for Romania pe cezicelegea.ro. Sistemul folosește o arhitectură RAG (Retrieval-Augmented Generation) construită deasupra unui model lingvistic mare (LLM) cuantizat, executat local pe hardware de larg consum, fără apeluri către servicii externe. Spre deosebire de lucrările existente în domeniu, contribuția centrală este un strat de verificare a ieșirii: răspunsul modelului este forțat să se conformeze unei scheme tipizate, iar un set de reguli verifică, după generare, prezența și validitatea citațiilor și faptul că documentele menționate apar efectiv în surse, abordând astfel categoria de erori nesemnificate identificate ca lucrare viitoare în Gheorghe et al. (2026). Sunt prezentate arhitectura sistemului, alegerile de implementare și un cadru de evaluare a calității regăsirii, a fidelității răspunsurilor și a costului computațional, comparând niveluri de cuantizare și scări de model.

Abstract

This thesis presents a conversational assistant for Romanian citizens that answers questions about administrative procedures (birth, marriage, housing), grounded in the structured data published by the Code for Romania project at cezicelegea.ro. The system is built on a Retrieval-Augmented Generation (RAG) architecture on top of a quantized Large Language Model (LLM) running locally on commodity hardware, with no calls to external services. The central contribution, beyond data and domain choices, is an output-verification layer: the model's response is constrained to a typed schema, and a set of rules check, after generation, the presence and validity of citations and that any mentioned documents actually appear in the sources, addressing the class of unflagged errors named as future work by Gheorghe et al. (2026). The thesis describes the system architecture, implementation decisions, and an evaluation framework covering retrieval quality, answer faithfulness, and computational cost across quantization levels and model scale.

Cuprins

1	Introducere	5
1.1	Motivație și problemă	5
1.2	Obiective și contribuții	6
2	Preliminarii	7
2.1	RAG (Retrieval-Augmented Generation)	7
2.2	Reprezentări vectoriale și regăsire	7
2.3	Modele lingvistice locale	8
2.4	Verificarea ieșirii	9
3	Sistemul propus	10
3.1	Vedere de ansamblu	10
3.2	Date și segmentare	11
3.3	Regăsire și generare	12
3.4	Stratul de verificare a ieșirii	14
3.5	Pre-completarea formularelor și interfața	15
4	Rezultate	17
4.1	Setup și metrici	17
4.2	Rezultate principale	18
4.3	Stratul de verificare: erori nesemnificate prinse	19
4.4	Modul de eșec dominant: refuzul nejustificat	19
4.5	Evaluarea pre-completării formularelor	20

5	Concluzii	21
5.1	Discuție și limitări	22
	Bibliografie	23

Capitolul 1

Introducere

1.1 Motivație și problemă

Digitalizarea administrației publice este o prioritate europeană, urmărită în România prin platforme precum SPV-ul ANAF, Ghișeul.ro sau RO e-Factura. Beneficiul direct pentru cetățean rămâne însă limitat. Proceduri obișnuite, cum sunt înregistrarea unei nașteri, încheierea unei căsătorii sau închirierea unei locuințe, presupun interacțiuni cu mai multe instituții, reglementări care se modifică în timp și formulare specializate, fără o sursă unică de informare. Proiectul *cezicelegea.ro* (Code for Romania) structurează informația pe evenimente de viață, însă rămâne un site informativ. Cetățeanul identifică singur ramura din arborele decizional și completează formularele de la zero.

Asistenții conversaționali bazați pe modele lingvistice mari și pe RAG (Retrieval-Augmented Generation) pot transforma un astfel de corpus într-un dialog cu răspunsuri ancorate în surse. În sectorul public există însă două constrângeri importante: confidențialitatea, întrucât datele cetățeanului nu pot părăsi dispozitivul, și fidelitatea, întrucât un răspuns plauzibil, dar nesuținut de surse, induce cetățeanul în eroare. Lucrarea abordează astfel întrebarea: cum poate fi construit un asistent pentru proceduri administrative românești care oferă răspunsuri verificabile și rulează integral pe hardware de larg consum, fără apeluri externe?

1.2 Obiective și contribuții

Lucrarea își propune să construiască și să evalueze un pipeline RAG complet, executabil integral local, pentru trei evenimente de viață (naștere, căsătorie, locuire), care să ofere răspunsuri verificabile, ancorate în sursele oficiale. Elementul central urmărit este un strat de verificare a ieșirii care vizează exact categoria de erori nesemnificate indicată drept lucrare viitoare în Gheorghe et al. (2026) [9]. În acest scop, lucrarea aduce următoarele contribuții:

- **Strat de verificare a ieșirii:** generarea este constrânsă la o schemă JSON tipizată, iar un set de reguli verifică, după generare, prezența și validitatea citațiilor și faptul că documentele menționate apar în surse.
- **Cadru de evaluare reproductibil** al sistemului: pe un set de aur de întrebări, sunt măsurate calitatea regăsirii, fidelitatea răspunsurilor și costul computațional, comparând niveluri de cuantizare, scări de model și un model dintr-o altă familie, specializat pe limba română.
- **Pre-completarea formularelor** printr-un motor condus de specificații, care acoperă un catalog de formulare administrative și extrage automat câmpurile dintr-o conversație în limbaj natural.
- **Arhitectură integral locală**, pe hardware de larg consum, motivată de constrângerile de confidențialitate din sectorul public.

Față de lucrările grupului de cercetare (Gheorghe et al. [9]; Paduraru et al. [17]), care propun arhitecturi multi-agent și injecție de erori, contribuția distinctă a acestei lucrări este implementarea și evaluarea concretă a stratului de verificare a ieșirii, pe o sursă de date și evenimente disjuncte, cu execuție locală.

Capitolul 2

Preliminarii

2.1 RAG (Retrieval-Augmented Generation)

Un model lingvistic mare generează răspunsuri pe baza tiparelor învățate la antrenare, fixate în parametrii săi. Pentru informație absentă din datele de antrenare, modelul fie semnalează că nu o deține, fie generează un răspuns plauzibil, dar nesusținut, cu aceeași certitudine ca unul corect (halucinație [10]). În domenii cu miză ridicată (juridic, medical, administrativ), un astfel de răspuns fals poate fi mai dăunător decât recunoașterea lipsei de informație.

RAG (Retrieval-Augmented Generation) [12] atenuează acest risc schimbând sursa răspunsului: modelul nu mai generează din cunoașterea parametrică, ci pe baza unor documente regăsite în prealabil. Procesul cuprinde două etape: regăsirea, în care sistemul selectează automat fragmentele cele mai relevante pentru întrebare, și generarea, în care răspunsul este compus pe baza acelor fragmente, oferite modelului ca context. Răspunsul devine astfel ancorat în surse, poate fi însoțit de citații verificabile, iar riscul de halucinație scade [8]. În plus, RAG separă cunoașterea (documentele) de capacitatea de a genera text, astfel încât actualizarea informației nu necesită reantrenarea modelului.

2.2 Reprezentări vectoriale și regăsire

Abordarea clasică de regăsire este cea lexicală, în care un fragment este relevant dacă împărtășește cuvinte cu întrebarea, ponderate după frecvență (familia BM25 [21]). Metoda este rapidă,

dar compară forme de cuvinte, nu sensuri: o întrebare despre „acte necesare” nu se potrivește cu un text despre „documente obligatorii”, deși înțelesul este același (nepotrivire de vocabular). În limba română problema se accentuează din cauza flexiunii bogate, care multiplică formele de suprafață ale aceluiași cuvânt.

Soluția este compararea sensului. Pornind de la observația că două cuvinte cu contexte asemănătoare au înțelesuri asemănătoare, cuvintele au fost reprezentate prin vectori denși (de exemplu word2vec [15]); modelele neuronale ulterioare au extins ideea la fraze și fragmente întregi, transformând un text într-un singur vector, numit embedding. Regăsirea devine astfel semantică (dense retrieval [11]): întrebarea și fragmentele sunt transformate în embeddings de același model (de exemplu multilingvul BGE-M3 [2]), iar relevanța este dată de apropierea vectorilor, măsurată prin similaritate cosinus. Pentru a evita compararea cu toți vectorii, regăsirea folosește o indexare aproximativă a vecinilor, de exemplu HNSW [13], oferită de baze de date vectoriale precum Chroma [3].

2.3 Modele lingvistice locale

Confidențialitatea datelor impune adesea ca modelul să ruleze local, fără transmiterea acestora către servicii externe; principala constrângere devine atunci memoria, determinată de numărul de biți alocați fiecărui parametru. Implicit, un parametru ocupă 32 de biți (FP32), deci un model de 3 miliarde de parametri necesită ~ 12 GB, prea mult pentru un calculator obișnuit. Cuantizarea [7] reduce numărul de biți per parametru: modelul ocupă mai puțin și se evaluează mai rapid, în schimbul preciziei. Nivelurile uzuale pentru un model 3B sunt:

- FP16 (16 biți): aproximativ jumătate din memorie (~ 6 GB), cu pierdere de precizie practic neglijabilă;
- Q8 (8 biți întregi): ~ 3 GB, cu degradare redusă a calității;
- Q4 (4 biți întregi): ~ 2 GB, cea mai agresivă comprimare uzuală, cu risc real de degradare a raționamentului.

Schemele k-quants din `llama.cpp` (sufixul `_K_M`) nu comprimă uniform, ci păstrează la

precizie mai mare parametrii importanți, obținând un compromis mai bun. Astfel de modele se rulează local prin `llama.cpp` (format GGUF) sau prin interfețe precum Ollama [16]. Pe lângă cuantizare, modelele locale se diferențiază prin scară (numărul de parametri: modelele mai mari raționează mai fidel, dar cer mai multe resurse) și prin specializare lingvistică: un model generalist multilingv (Qwen2.5 [23]) acoperă multe limbi, în timp ce unul antrenat special pe o limbă cu resurse reduse (RoLlama3 [14]) poate compensa subreprezentarea ei.

2.4 Verificarea ieșirii

În contexte reglementate, generarea de text liber nu este suficientă: răspunsul trebuie să poată fi verificat. Gheorghe et al. (2026) [9] arată empiric că aproximativ 63% dintre erorile injectate la limitele serviciilor externe rămân nesemnificate (modelul produce răspunsuri plauzibile pe date incomplete) și propun verificarea sistematică a ieșirii ca direcție de cercetare. Un strat de verificare a ieșirii are două componente: o schemă structurată (un format tipizat, de exemplu JSON, impus motorului de inferență, care elimină erorile de formă) și un set de reguli aplicate după generare, care verifică proprietăți de conținut, precum existența și validitatea citațiilor și prezența în surse a documentelor menționate. La eșec, generarea poate fi reluată cu feedback explicit, iar la epuizarea încercărilor răspunsul devine un refuz controlat.

Verificarea externă se distinge de cea integrată în model (Self-RAG [1]) și se înrudește cu guardrails programabile (NeMo Guardrails [19]) și cu cadre de evaluare a RAG (RAGAS [5]); spre deosebire de un model-judecător extern, regulile de verificare sunt deterministe, auditabile și rulează integral local.

Capitolul 3

Sistemul propus

3.1 Vedere de ansamblu

Sistemul implementează un pipeline RAG complet pentru cele trei evenimente de viață de pe ceziclegea.ro [4] (naștere, căsătorie, locuire), rulând local după descărcarea inițială a modelelor. Se compune din șase module Python: **scraper**, **chunker** (segmentare cu breadcrumb), **index** (embedding + Chroma), **retriever**, **generator** (LLM cu constrângere de schemă) și **contracts** (reguli de verificare a ieșirii).

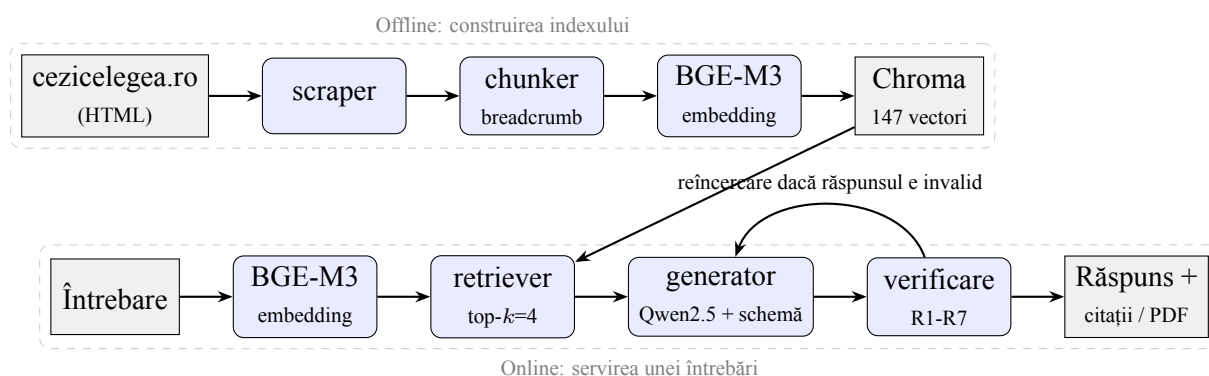


Figura 3.1: Arhitectura sistemului, cu faza offline de construire a indexului și faza online de servire a unei întrebări.

3.2 Date și segmentare

Conținutul de pe cezicelegea.ro este o aplicație de tip state machine pe partea de client (Alpine.js): întreg arborele de decizie al fiecărui eveniment este prezent în HTML-ul static, dar vizibilitatea fiecărui nod este controlată în browser. Scraperul (`requests` + `BeautifulSoup` [20], fără motor de browser) parsează DOM-ul static și reconstruiește offline arborele real: îl parcurge în pre-ordine (DFS) cu o stivă de context, luând conținutul nodului din titlurile h2 și etichetele copiilor din h3-urile butoanelor de navigare. Deoarece etichetele sunt uneori prescurtate, potrivirea părinte-copil folosește o similaritate Jaccard pe cuvinte (prag 0.7). Rezultă 107 secțiuni cu breadcrumb complet (până la 8-9 niveluri), cu cele trei ramuri principale reconstruite corect.

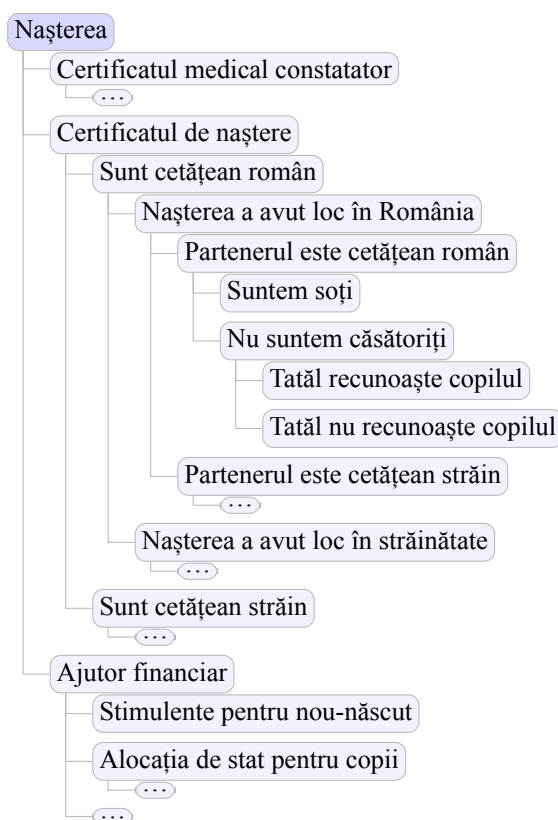


Figura 3.2: Fragment simplificat din arborele de decizie al evenimentului Nașterea, reconstruit offline din DOM-ul static (subarborii marcați cu ... sunt omiși). Arborele real atinge 8-9 niveluri; fiecare frunză primește un breadcrumb complet, prefixat în textul indexat.

Modulul `chunker` prefixează breadcrumb-ul în textul indexat, sub forma [Nașterea >

Certificatul de naștere > ... > Tatăl recunoaște copilul] urmată de conținut (Figura 3.2). Prefixul contribuie la embedding și distinge ramuri semantice apropiate (de exemplu, documente pentru cuplu căsătorit versus pentru un părinte necăsătorit), păstrând trasabilitatea sursei. Secțiunile mai mari de ~ 3000 de caractere sunt împărțite greedy la limite de propoziție; indexul final conține 147 de fragmente.

3.3 Regăsire și generare

Embedding-urile sunt calculate cu BGE-M3 [2] (multilingv, 1024 de dimensiuni), normalizate L2 astfel încât similaritatea cosinus din Chroma să se reducă la produs scalar. La interogare, baza returnează cele mai relevante $k = 4$ fragmente. Valoarea este un compromis (un k prea mic ratează fragmentul cu răspunsul, iar unul prea mare diluează contextul și crește costul), iar la $k = 4$ regăsirea acoperă deja aproape integral fragmentele relevante (Capitolul 4). Cele trei evenimente formează grupuri bine separate în spațiul de embedding, ceea ce explică recall-ul ridicat al regăsirii (Figura 3.3).

Proiecție PCA 3D a fragmentelor (BGE-M3, 1024 $\rightarrow$ 3)

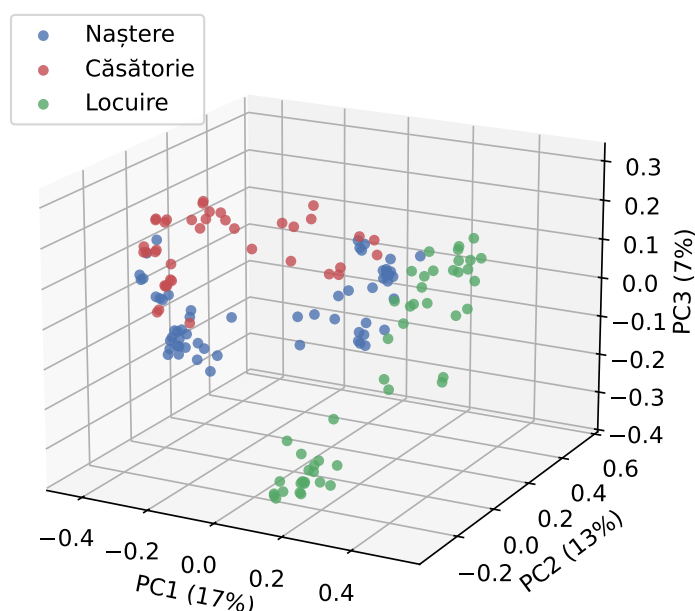


Figura 3.3: Fragmentele celor trei evenimente, separate în spațiul de embedding.

Generarea rulează integral local prin Ollama (peste llama.cpp, format GGUF); configurația

implicită este Qwen2.5-3B-Instruct [23], suficient de mică pentru hardware de larg consum. Apelul conține un prompt de sistem care impune limba română și ancorarea strictă în surse, un mesaj de utilizator cu sursele numerotate [S1] ...[Sn] și întrebarea, plus o schemă JSON derivată din clasa Pydantic [18] AnswerSchema, transmisă prin parametrul format pentru a forța conformitatea structurală.

Promptul de sistem (redat condensat mai jos) codifică regulile de comportament pe care stratul de verificare a ieșirii le controlează ulterior automat:

Ești un asistent care ajută cetățenii români să înțeleagă proceduri administrative (naștere, căsătorie, locuire). Respectă strict următoarele reguli:

- (1) Răspunde exclusiv în limba română.
- (2) Bazează-te numai pe SURSELE furnizate; nu inventa documente, termene, instituții sau cifre.
- (3) Indică sursele folosite prin indici (de exemplu, sursele 1 și 3 -> `cited_source_indices=[1,3]`).
- (4) Dacă SURSELE nu conțin răspunsul, setează `is_refusal=true` și răspunde: „Nu am suficiente informații în sursele disponibile pentru a răspunde.”
- (5) În `documents_mentioned` trece doar numele reale de documente care apar explicit în SURSE.
- (6) Formulează răspunsul concis și structurat.
- (7) Setează `form_offer` numai dacă utilizatorul își exprimă intenția de a completa un formular.

Schema răspunsului are cinci câmpuri: `answer_text`, `is_refusal`, `cited_source_indices`, `documents_mentioned` și `form_offer`. Ultimele două sunt esențiale pentru regulile care împiedică inventarea de documente, respectiv pentru tranziția către pre-completarea formularelor.

3.4 Stratul de verificare a ieșirii

După obținerea răspunsului structurat, sistemul aplică reguli automate care verifică nu forma (garantată de schemă), ci conținutul: măsura în care răspunsul se sprijină pe surse. Fiecare regulă vizează un mod în care un răspuns poate părea corect, dar poate induce cetățeanul în eroare:

- **R1: existența unui răspuns.** Verifică faptul că schema conține un text de răspuns nevid (câmpul `answer_text`). Motivație: prinde eșecurile complete de generare (răspuns gol, ieșire degenerată sau generare întreruptă), care altfel ar fi servite cetățeanului ca răspuns valid. Această regulă este totodată o precondiție pentru celelalte reguli, întrucât fără text nu există ce cita sau ancora.
- **R2: citarea afirmațiilor.** Un răspuns care nu este refuz trebuie să indice cel puțin o sursă. Motivație: în lipsa citației, informația nu poate fi verificată, iar un fapt real nu poate fi distins de unul inventat.
- **R3: validitatea citațiilor.** Indicii citați trebuie să existe efectiv în lista de surse (între 1 și n). Motivație: o trimitere la „sursa 7” atunci când există doar 4 surse reprezintă o citație inventată.
- **R4: ancorarea documentelor menționate.** Orice document numit în răspuns trebuie să se regăsească în textul sursei citate. Motivație: este principala regulă care prinde documentele inventate, identificând situațiile în care modelul indică un act sau un formular ce nu există în procedură.
- **R5: refuzul nu menționează documente.** Atunci când modelul refuză să răspundă, nu are voie să enumere documente. Motivație: un refuz însoțit de documente reprezintă o invenție mascată: pare prudent, dar introduce informație nesusținută.
- **R6: absența identificatorilor interni de formular.** Identificatorii tehnici (de exemplu `declaratie_nume_copil`) nu trebuie confundați cu nume reale de acte. Motivație: se evită expunerea catalogului intern de formulare în răspunsul oferit cetățeanului.

Regulile R1-R6 constituie setul evaluat în Capitolul 4. Ulterior a fost adăugată regula **R7**, care semnalează un refuz nejustificat: modelul refuză, deși o sursă regăsită cu similaritate ridicată conține răspunsul. Motivație: este singura regulă care vizează modul de eșec dominant, în care cetățeanul primește un refuz la o întrebare legitimă. Acest mod de eșec este invizibil pentru R1-R6, deoarece un refuz nu conține nici citații, nici documente care să poată fi verificate.

Limba de răspuns nu este verificată printr-o regulă separată, deoarece este deja impusă de promptul de sistem și a fost respectată de toate modelele evaluate. Regulile R1-R7 se concentrează pe modul de eșec relevant pentru fidelitate: ancorarea răspunsului în surse.

Atunci când o regulă nu este satisfăcută și mai există încercări disponibile, sistemul reapelează modelul cu un mesaj de feedback explicit. La epuizarea încercărilor (implicit una), răspunsul este marcat drept invalid, iar interfața afișează un refuz controlat.

În testarea manuală s-au observat trei comportamente. Pentru întrebări clare, sistemul produce răspunsuri ancorate în surse. Pentru întrebări care combină mai multe evenimente, regăsirea este dominată de termenul cu pondere mai mare, iar modelul refuză în loc să fabrice. Pentru întrebări din afara domeniului, modelul tinde să genereze o procedură generică, pe care verificarea o identifică prin R2 sau R4.

3.5 Pre-completarea formularelor și interfața

Sistemul include un strat de pre-completare a formularelor, declanșat atunci când utilizatorul își exprimă intenția (prin câmpul `form_offer` și un detector pe cuvinte-cheie). Stratul este condus de specificații: fiecare formular este declarat o singură dată, ca listă de câmpuri tipizate, din care se derivă automat schema Pydantic, validatorul (CNP, serie și număr CI, dată ISO 8601, IBAN), randarea PDF (`fpdf2` [6]) și extractorul; astfel, adăugarea unui formular nou se reduce la o descriere declarativă, fără cod suplimentar. Catalogul acoperă șapte formulare pe cele trei evenimente. Extragerea folosește o schemă JSON partiționată pe secțiuni (3-6 câmpuri), pentru a evita omiterea câmpurilor; la eșec, sistemul comută la un mod manual cu validare prin expresii regulate, garantând producerea unui PDF semnabil.

CERERE

pentru acordarea alocației de stat pentru copii

Subsemnatul Ion Popescu, CNP 1900101227783, domiciliat în str. Florilor nr. 5, ap. 3, sector 2, București, CI seria RX nr. 123456, solicit acordarea alocației de stat pentru copilul Andrei Popescu, CNP 5230105227781, născut la data de 2026-05-25.

Solicit virarea alocației în contul IBAN RO49AAAA1B31007593840000.

Data: 10.06.2026
Locul: București

Semnătura solicitantului: _____

Figura 3.4: Antetul și corpul unui formular generat de stratul de pre-completare (cererea pentru alocația de stat), pornind de la datele extrase dintr-o conversație în limbaj natural.

Interfața web (Streamlit [22]) afișează conversația, sursele citate marcate distinct și butoanele de descărcare a formularelor, deschizând la nevoie blocul de pre-completare cu intrare generată de model sau introdusă manual (Figura 3.5). Interfața folosește aceeași infrastructură ca varianta în linie de comandă, ceea ce asigură un comportament unic, derivat dintr-o singură sursă de adevăr.

Întrebări recente

+ Întrebare nouă

► Cerere alocație copil

Surse

★ = sursă citată în ultimul răspuns

★ S1 — Nașterea > Alocația de stat pentru copii > Acte necesare
<https://coziceleaga.ro/nastere>

★ S2 — Nașterea > Alocația de stat pentru copii > Depunerea cererii
<https://coziceleaga.ro/nastere>

• S3 — Nașterea > Înregistrarea nașterii
<https://coziceleaga.ro/nastere>

• S4 — Locuire > Despre contractul de locațiune
<https://coziceleaga.ro/loaiciune>

Formulare generate

↓ cerere_alocatie_copii.pdf

Asistent pentru proceduri administrative

Vreau să completez o cerere pentru alocația copilului meu.

Pentru a completa cererea pentru alocația de stat pentru copii, veți avea nevoie să depuneți următoarele documente: certificatul de naștere al copilului, actele de identitate ale părinților (cartea de identitate sau pașaport), certificatul de căsătorie dacă este cazul și extrasul de cont bancar pentru plata prin transfer.

Documente menționate:

- certificat de naștere al copilului
- act de identitate
- extras de cont bancar

Cerere pentru alocația de stat pentru copii

Cerere pentru acordarea alocației de stat pentru copii, depusă de unul dintre părinți după înregistrarea nașterii.

11 din 11 câmpuri completate

Vezi câmpurile colectate

Solicitant

- ✓ Numele complet (solicitant): Ion Popescu
- ✓ CNP (solicitant): 1850412220011
- ✓ Adresa de domiciliu (solicitant): Str. Florilor 12, București
- ✓ Seria CI (solicitant): RT
- ✓ Numărul CI (solicitant): 234567

Figura 3.5: Interfața web: în bara laterală, istoricul de întrebări și panoul Surse (sursele citate marcate distinct); în centru, dialogul cu modelul și blocul de pre-completare a formularelor.

Capitolul 4

Rezultate

4.1 Setup și metrice

Sistemul este evaluat pe un set de aur de 98 de cazuri (35 naștere, 30 căsătorie, 33 locuire), dintre care 16 de tip refuz controlat (în afara domeniului); proveniența și validarea sa sunt discutate în Secțiunea 5.1. Toate rulările folosesc aceiași parametri ($k = 4$, o singură reîncercare, temperatura 0) și aceeași infrastructură (GPU $2 \times$ Tesla T4). Comparația cantitativă principală este un **studiu comparativ** a șase configurații: modelul 3B la trei nive-uri de cuantizare (FP16, Q8, Q4) și două scări mai mari la Q4 (7B și 14B), plus un model RO-specializat din altă familie (RoLlama3-8B), rulat prin pipeline-ul complet. Codul sursă, setul de aur și artefactele de evaluare sunt disponibile public la <https://github.com/cosmin-glod/Romanian-Legal-Document-Assistent>, pentru reproductibilitate.

Se calculează cinci metrice: refusal accuracy (`is_refusal` vs `should_refuse`), breadcrumb recall@k, lexical keyword recall (potrivire exactă de sub-șir a cuvintelor-cheie așteptate), rată de validare (cazuri care trec toate regulile R1-R6) și latency. Recall-ul de breadcrumb și cel de cuvinte-cheie sunt proxy-uri: al doilea este o limită inferioară a fidelității (nu recunoaște sinonime), triangulată mai jos printr-un eșantion evaluat manual.

Config	Refusal	Recall	Lexical	Validare	Fidelit.	Lat. (s)
3B-Q4	62	95	41	53	76	13
3B-Q8	74	95	53	68	86	16
3B-FP16	79	95	55	68	83	27
7B-Q4	69	95	57	68	92	28
14B-Q4	66	95	47	71	100	37
RoLlama-8B	82	95	71	56	79	48

Tabela 4.1: Rezultate pe setul de aur: cuantizare, scară și model RO-specializat.

4.2 Rezultate principale

Tabelul 4.1 sintetizează rezultatele celor șase configurații: modelul 3B la trei niveluri de cuantizare (FP16, Q8, Q4), două scări mai mari la Q4 (7B și 14B) și modelul RO-specializat RoLlama3-8B; fidelitatea răspunsurilor este evaluată separat de un model judecător de 14B.

Se remarcă trei aspecte principale. Întâi, cuantizarea la 4 biți are un cost real: 3B-Q4 este cea mai slabă configurație de 3B la refuz, acoperire lexicală și validare, în timp ce 3B-Q8 se apropie de 3B-FP16 la o fracțiune din memorie. Apoi, precizia poate compensa scara: 3B-Q8 egalează sau depășește la refusal accuracy modelele mai mari 7B-Q4 și 14B-Q4, deci un model mic, bine cuantizat, este competitiv cu unul mare slab cuantizat. În al treilea rând, fidelitatea (judecată de modelul de 14B) crește monoton cu scara, în timp ce recall-ul rămâne constant pe toate configurațiile, ceea ce confirmă că factorul determinant este generarea, nu regăsirea. Judecătorul nu a marcat aproape niciun răspuns drept incorect, deci valorile de fidelitate sunt o limită superioară optimistă, triangulată de eșantionul evaluat manual mai jos.

Schimbarea familiei de model aduce o observație distinctă. RoLlama3-8B, specializat pe limba română, reduce drastic refuzul nejustificat (3 cazuri, față de 33 pe 3B-Q4), ceea ce confirmă că specializarea lingvistică acționează direct asupra modului de eșec dominant (Secțiunea 4.4). Compromisul constă într-o deplasare către eșecul opus, sub-refuzul (modelul răspunde la 15 din 16 întrebări din afara domeniului), și într-o disciplină mai slabă de citare și ancorare, surprinsă de stratul de verificare prin încălcări ale R2, R4 și R6. Cu alte cuvinte, un model specializat pe română deplasează profilul de eroare dinspre supra-refuz către sub-refuz și erori de ancorare, adică tocmai zona în care verificarea ieșirii aduce valoare.

Pentru a calibra lexical keyword recall (41% pe 3B), 15 răspunsuri non-refuz au fost evaluate

manual: 27% corecte, 47% parțiale, 27% incorecte. Metrica lexicală subevaluează deci calitatea, însă fidelitatea pe 3B rămâne modestă, eroarea recurentă fiind nu inventarea, ci confuzia de rol (proprietar $\leftrightarrow$ chiriaș, drepturi $\leftrightarrow$ obligații), pe care verificarea nu o identifică.

4.3 Stratul de verificare: erori nesemnalate prinse

Rolul principal al verificării este detectarea răspunsurilor plauzibile, dar nesuținute de surse, adică a erorilor nesemnalate descrise în [9]. Regula R4 (documente inventate) domină încălcările pe 3B (26 de apariții) și se prăbușește odată cu scara (2 pe 7B, 0 pe 14B), în timp ce R5 (refuz însoțit de documente inexistente) apare doar pe 7B (8 apariții). Astfel, creșterea scării schimbă clasa de eșec, surprinsă de aceeași infrastructură. Precizia regulii R4 a fost auditată manual: din 10 cazuri semnalate, 9 reprezintă probleme reale ($\approx 90\%$ precizie).

O ablație pe configurația Qwen2.5-3B-Q4 măsoară direct valoarea stratului: în absența verificării, **66 din 98** de răspunsuri (67%) ar fi fost servite cu defect, în majoritate din cauza lipsei citațiilor (60 de răspunsuri neverificabile). Reîncercarea **repară 48** dintre ele și **reține 18** sub forma unui refuz controlat, astfel încât niciun răspuns defect nu ajunge la cetățean. Structural, citarea (R2) este o precondiție pentru R4: în prima încercare aproape niciun răspuns nu conține citații, astfel încât documentele inventate (R4) devin vizibile abia după ce reîncercarea adaugă citațiile.

4.4 Modul de eșec dominant: refuzul nejustificat

Cea mai frecventă eroare nu este inventarea de informație, ci refuzul nejustificat: modelul răspunde „Nu am suficiente informații” la o întrebare al cărei răspuns este, de fapt, prezent în surse. Pe 3B-Q4, **33 din 82** de întrebări din domeniu (40%) primesc un refuz nejustificat, dintre care 31 aveau răspunsul regăsit în top- k ; pe 7B-Q4, **28 din 82** (34%). Refuzul nejustificat depășește inventarea de documente și rămâne modul de eșec dominant. Scara nu îl reduce semnificativ (14B-Q4 păstrează 30 de cazuri), în schimb o cuantizare de calitate da (3B-Q8 coboară la 20). Un prompt reformulat recuperează doar 13 din 33 de cazuri, deci eșecul ține de calibrarea generării, nu de regăsire.

Prin construcție, stratul de verificare nu poate detecta acest mod de eșec: un refuz trece de R1-R6 fără să conțină vreo citație sau document de verificat. Această limitare a motivat regula **R7**, care semnalează un refuz suspect atunci când similaritatea top-1 depășește un prag. Calibrat pe setul de aur, pragul 0.64 separă refuzurile corecte (≤ 0.63) de cele nejustificate (până la 0.80); R7 detectează **24 din 33** de cazuri (73%) cu **zero fals pozitive**, fiind mai precis decât reformularea promptului și independent de generare. În practică, regula este folosită pentru semnalarea cazurilor în vederea revizuirii, nu pentru recuperarea automată.

Pentru ilustrare, mai jos sunt date un răspuns corect și un eșec tipic, ambele din rularea pe 7B-Q4:

Răspuns corect (ancorat). Întrebare: „Ce documente îmi trebuie pentru înregistrarea nașterii copilului, dacă sunt căsătorit cu partenera?” Răspuns: „Actele de identitate ale părinților; Certificatul medical constatator al nașterii...”, cu citarea surselor S1 și S3.

Refuz nejustificat. Întrebare: „Cine poate primi indemnizația pentru creșterea copilului?” Răspuns: „Nu am suficiente informații în sursele disponibile pentru a răspunde.”, deși răspunsul se afla în fragmentul regăsit cu cea mai mare similaritate.

4.5 Evaluarea pre-completării formularelor

Stratul de pre-completare a fost evaluat pe 14 conversații (două per formular, 180 de câmpuri), cu o singură rundă de extracție deterministă. Acuratețea globală este **99%** (179/180): șase din șapte formulare ating 100%, singurul sub acest prag fiind contractul de închiriere (97%). Un pas determinist suplimentar (expresii regulate pentru date în format ISO sau românesc și deducerea sexului din „băiat”/„fetiță”) a crescut acuratețea de la 94% la 99% și rata de completare dintr-o singură rundă de la 57% la 93%. Singurul câmp ratat provine dintr-o confuzie de rol (adresa proprietarului în câmpul imobilului); deoarece niciun câmp nu este inventat (lipsă → null → completare manuală), fiecare formular conduce la un PDF complet.

Capitolul 5

Concluzii

Lucrarea a propus și a evaluat un pipeline complet, executabil local, pentru un asistent conversațional dedicat procedurilor administrative românești, ancorat în datele publicate de Code for Romania. Față de lucrările grupului de cercetare (Gheorghe et al., 2026 [9]; Paduraru et al., 2026 [17]), contribuțiile principale sunt: o sursă de date și evenimente de viață disjuncte; un strat de verificare a ieșirii (șase reguli evaluate, plus R7 pentru supra-refuz) care implementează verificarea răspunsului identificată în literatură ca lucrare viitoare; și o arhitectură integral locală, pe hardware de larg consum.

Evaluarea cantitativă susține valoarea contribuțiilor: pe 3B, **67%** dintre răspunsuri ar fi fost servite cu defect fără stratul de verificare (în majoritate din lipsa citațiilor), iar reîncercarea le repară sau le reține controlat. Trecerea la modelul mai mare reduce documentele inventate de la 26 de cazuri la 2 (o reducere de peste 90%), validând un mecanism de verificare independent de model.

Un rezultat al evaluării clarifică o alegere de proiectare: studiul comparativ de cuantizare arată că principalul factor de degradare este precizia, nu scara. Configurația 3B-Q8 (74% refusal accuracy) depășește 3B-Q4 (62%) și întrece modele mai mari, dar slab cuantizate (7B-Q4 69%, 14B-Q4 66%), deci un model mic, bine cuantizat, este competitiv pentru rularea locală. Analiza modurilor de eșec a condus și la regula R7 de detecție a supra-refuzului, care, calibrată pe setul de aur, detectează 73% dintre refuzurile nejustificate fără fals pozitive.

Experimentul cu un model specializat pe limba română (RoLlama3-8B) confirmă o direcție și deschide alta: specializarea reduce drastic refuzul nejustificat (3 cazuri, față de 33), dar

deplasează eroarea către sub-refuz și o citare mai slabă, ceea ce justifică valoarea stratului de verificare. Două direcții rămân prioritare pentru dezvoltarea ulterioară: o regăsire conștientă de intenție, pentru întrebările care combină mai multe evenimente, și o regulă suplimentară R8 de ancorare cross-eveniment, integrabilă în stratul de verificare fără modificarea logicii de generare.

5.1 Discuție și limitări

Verificarea adresează eficient eșecul zgomotos (inventarea de documente), însă modul de eșec dominant, refuzul nejustificat, rămâne în mare parte invizibil pentru R1-R6 și este doar semnalat de R7. Prin urmare, valoarea verificării este reală, dar parțială.

Câteva limitări țin de validitatea de construct: breadcrumb recall@k și lexical keyword recall sunt proxy-uri, triangulate cu o evaluare manuală (15 cazuri), cu un judecător LLM și cu auditul R4 ($\approx 90\%$ precizie), iar cuvintele-cheie au fost validate automat ca prezente în corpusul-sursă; în plus, „eroarea nesemnaltă” este o definiție operațională (declanșarea unei reguli), nu o judecată independentă.

Din perspectiva validității interne, recall-ul constant izolează diferențele la nivelul generării, însă pragul R7 este calibrat și evaluat pe același set de 98 de cazuri, deci performanța de detecție este in-sample, validarea pe un set ținut deoparte rămânând lucrare viitoare; întrucât toate configurațiile au fost evaluate pe aceeași infrastructură, diferențele de calitate sunt atribuibile exclusiv modelului.

În ceea ce privește validitatea externă, setul de aur (98 de cazuri, generat asistat de LLM) acoperă doar trei evenimente de viață, un singur model de embedding și exclusiv limba română, pe un instantaneu al corpusului, astfel încât generalizarea la alte domenii sau limbi nu este garantată.

Bibliografie

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil și Hannaneh Hajishirzi, „Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”, în The Twelfth International Conference on Learning Representations (ICLR 2024), 2024, DOI: [10.48550/arXiv.2310.11511](https://doi.org/10.48550/arXiv.2310.11511).
- [2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian și Zheng Liu, „BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation”, în arXiv preprint arXiv:2402.03216 (2024), DOI: [10.48550/arXiv.2402.03216](https://doi.org/10.48550/arXiv.2402.03216).
- [3] Chroma, Chroma: the open-source embedding database, <https://www.trychroma.com>, Accesat: 7 iunie 2026.
- [4] Code for Romania, Ce zice legea, <https://cezicelegea.ro>, Accesat: 7 iunie 2026.
- [5] Shahul Es, Jithin James, Luis Espinosa-Anke și Steven Schockaert, „RAGAS: Automated Evaluation of Retrieval Augmented Generation”, în Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024): System Demonstrations, 2024, DOI: [10.48550/arXiv.2309.15217](https://doi.org/10.48550/arXiv.2309.15217).
- [6] fpdf2 contributors, fpdf2: Minimalist PDF creation library for Python, <https://py-pdf.github.io/fpdf2/>, Accesat: 7 iunie 2026.
- [7] Elias Frantar, Saleh Ashkboos, Torsten Hoefer și Dan Alistarh, „GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”, în The Eleventh International Conference on Learning Representations (ICLR 2023), 2023, DOI: [10.48550/arXiv.2210.17323](https://doi.org/10.48550/arXiv.2210.17323).

- [8] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang și Haofen Wang, „Retrieval-Augmented Generation for Large Language Models: A Survey”, în arXiv preprint arXiv:2312.10997 (2023), DOI: [10.48550/arXiv.2312.10997](https://doi.org/10.48550/arXiv.2312.10997).
- [9] Radu-Mihai Gheorghe, Petru-Liviu Bouruc, Ciprian Paduraru și Alin Stefanescu, „Robust Agentic AI for Public Administration: A Multi-Agent Architecture and Controlled Fault Injection Evaluation”, în Proceedings of the 30th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2026), Procedia Computer Science, Elsevier, 2026.
- [10] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto și Pascale Fung, „Survey of Hallucination in Natural Language Generation”, în ACM Computing Surveys 55.12 (2023), pp. 1–38, DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [11] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen și Wen-tau Yih, „Dense Passage Retrieval for Open-Domain Question Answering”, în Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020), 2020, pp. 6769–6781.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel și Douwe Kiela, „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, în Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020.
- [13] Yu A. Malkov și Dmitry A. Yashunin, „Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”, în IEEE Transactions on Pattern Analysis and Machine Intelligence 42.4 (2020), pp. 824–836, DOI: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [14] Mihai Masala, Denis C. Ilie-Ablachim, Dragos Corlatescu, Miruna Zavelca, Marius Leordeanu, Stefan Dumitrescu și Traian Rebedea, OpenLLM-Ro – Technical Report on Open-source Romanian LLMs trained starting from Llama 2, <https://arxiv.org/abs/2405.07703>, 2024, arXiv: [2405.07703](https://arxiv.org/abs/2405.07703) [cs.CL].

- [15] Tomas Mikolov, Kai Chen, Greg Corrado și Jeffrey Dean, „Efficient Estimation of Word Representations in Vector Space”, în 1st International Conference on Learning Representations (ICLR 2013), Workshop Track, 2013.
- [16] Ollama, Ollama: Get up and running with large language models locally, <https://ollama.com>, Accesat: 7 iunie 2026.
- [17] Ciprian Paduraru, Bogdan Dumitru și Alin Stefanescu, „A Guardrail-driven Multi-Agent Architecture for AI-Assisted Public Administration Workflows”, în Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2026), 2026.
- [18] Pydantic, Pydantic: Data validation using Python type hints, <https://docs.pydantic.dev>, Accesat: 7 iunie 2026.
- [19] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien și Jonathan Cohen, „NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails”, în Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023): System Demonstrations, 2023, DOI: [10.48550/arXiv.2310.10501](https://doi.org/10.48550/arXiv.2310.10501).
- [20] Leonard Richardson, Beautiful Soup Documentation, <https://www.crummy.com/software/BeautifulSoup/>, Accesat: 7 iunie 2026.
- [21] Stephen Robertson și Hugo Zaragoza, „The Probabilistic Relevance Framework: BM25 and Beyond”, în Foundations and Trends in Information Retrieval 3.4 (2009), pp. 333–389, DOI: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- [22] Streamlit, Streamlit: A faster way to build and share data apps, <https://streamlit.io>, Accesat: 7 iunie 2026.
- [23] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei et al., Qwen2.5: A Party of Foundation Models, <https://qwenlm.github.io/blog/qwen2.5/>, 2024.